

Graph

Operations

- Add vertex
 - Inserting a new vertex
- Delete vertex
 - Remove all connecting edge
- Add edge
- Delete Edge
- Find Vertex
- Traverse Graph
 - Depth-first Traversal
 - Breadth-first Traversal

Fig.11-4,5 Add/Delete Vertex

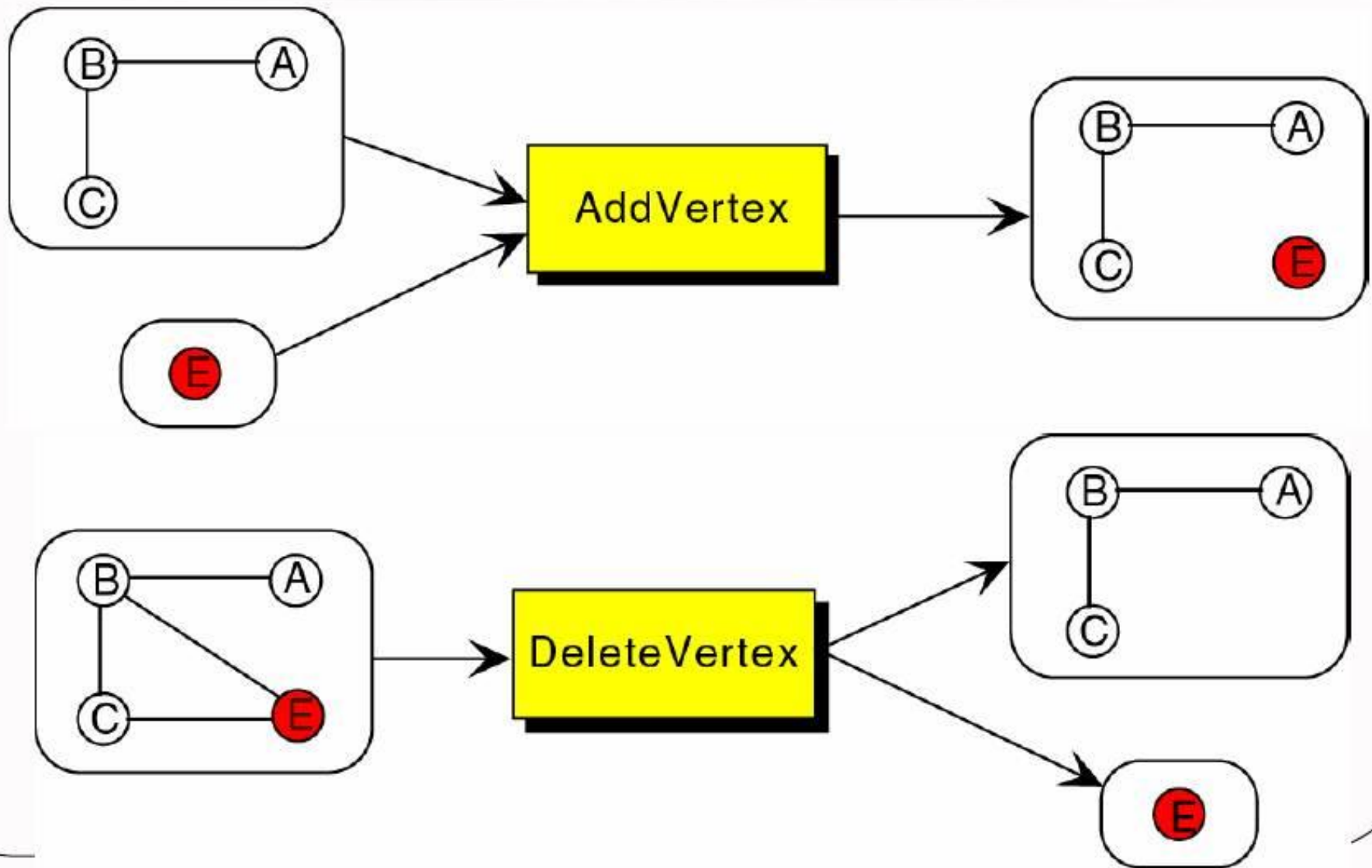
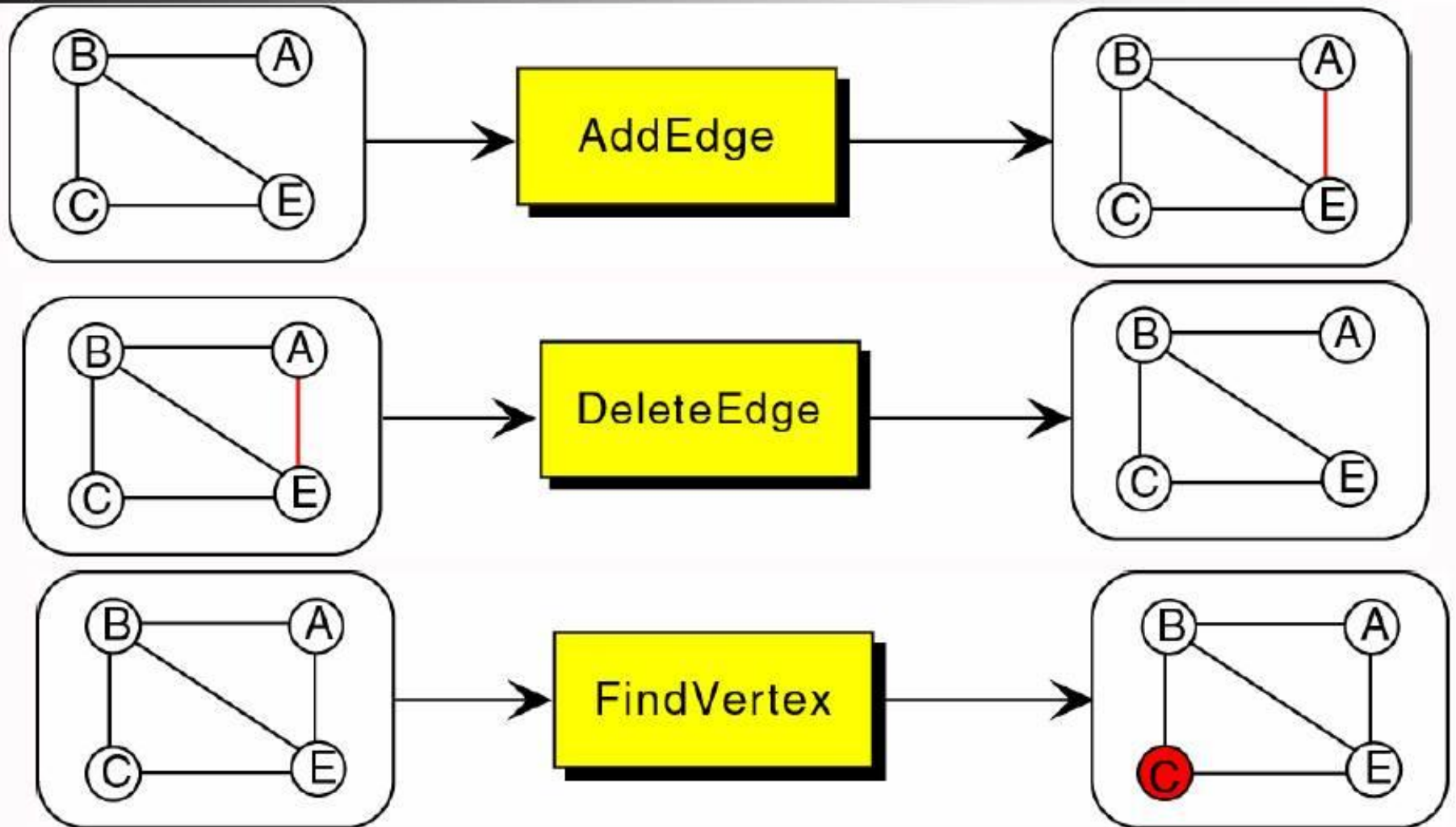


Fig.11-6,7,8



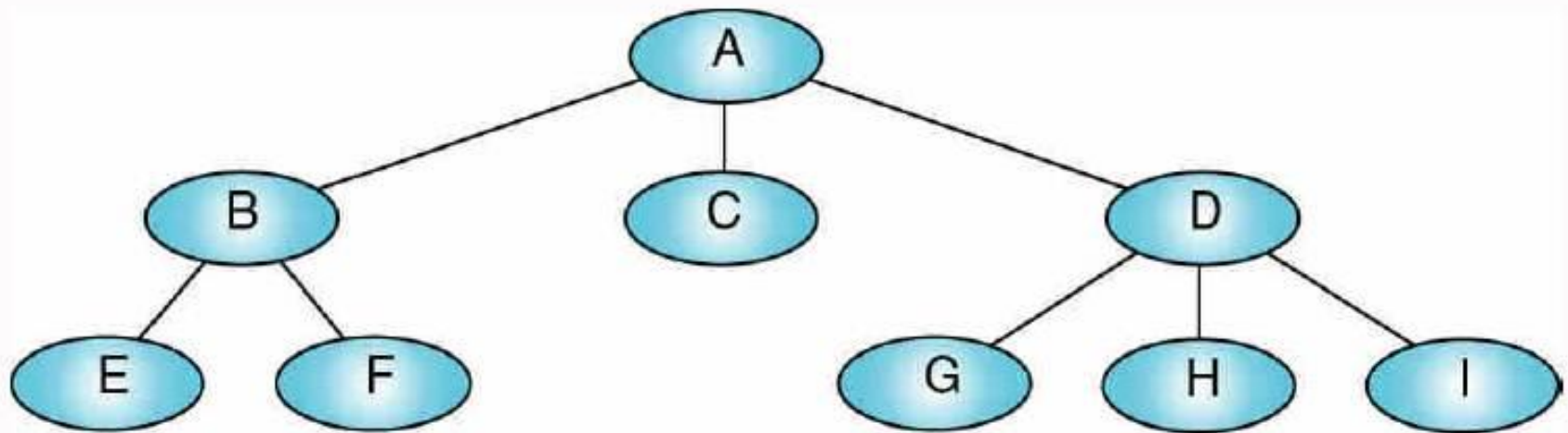
Traversals

- **New challenge** – a vertex in a graph having multiple parents
 - Possible to have different paths to a vertex
- **Goals** – to assure processing the data
 - To use a “visited flag”
 - Set all flags off initially
 - Set the flag of a vertex on when paying a visit

Depth-First Traversal

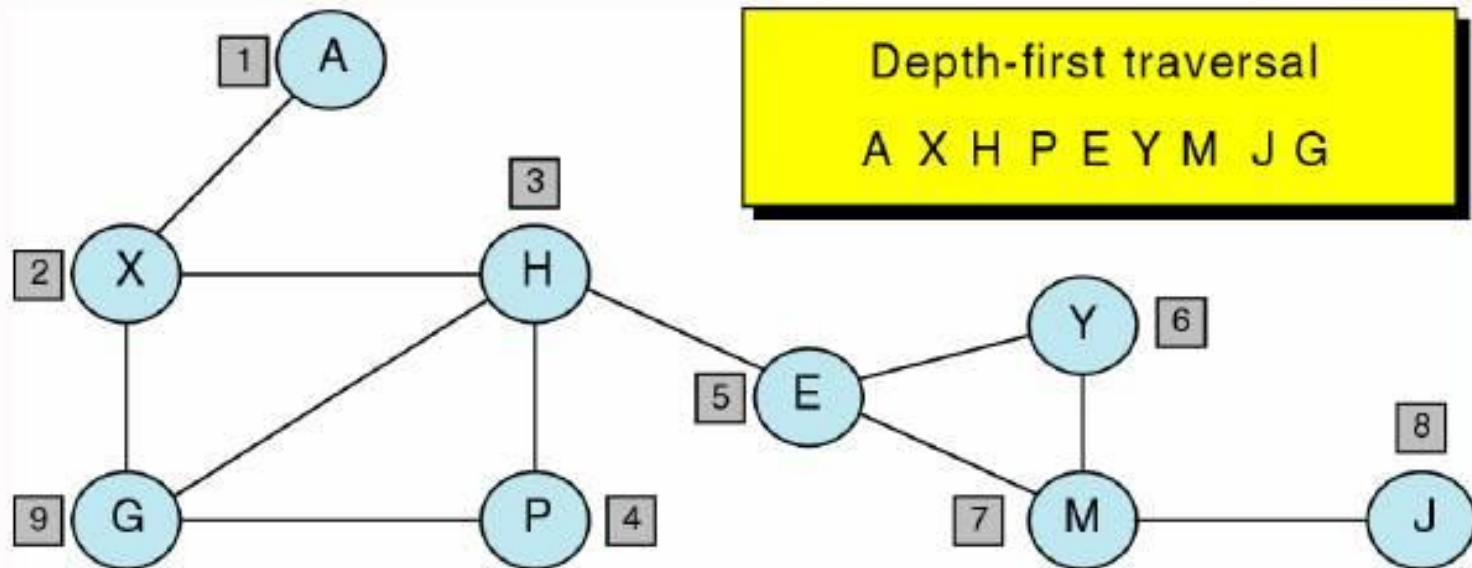
- All of a vertex's *descendants* are processed before we move to an adjacent vertex.
- Steps
 - Process the **first vertex** of the graph
 - Select **any vertex adjacent to the first vertex** and process
 - Select and process **any adjacent vertex** until reaching a vertex with no adjacent vertex and, then **backtrack**

Depth First Traversal on a Tree

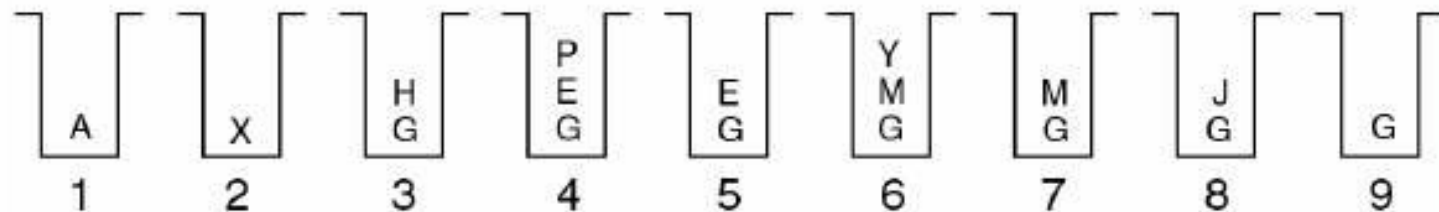


Depth-first traversal: A B E F C D G H I

Fig.11-10 DFT with a Stack



(a) The graph



(b) Stack contents

Breadth-First Traversal

- All adjacent vertices of a vertex are processed before going to the next level.
- Steps
 - Process the first vertex of the graph
 - Process all the of its adjacent vertices
 - Select the each adjacent vertex and process vertices adjacent it.
- Repeat ...

Breadth-First Traversal on a Tree

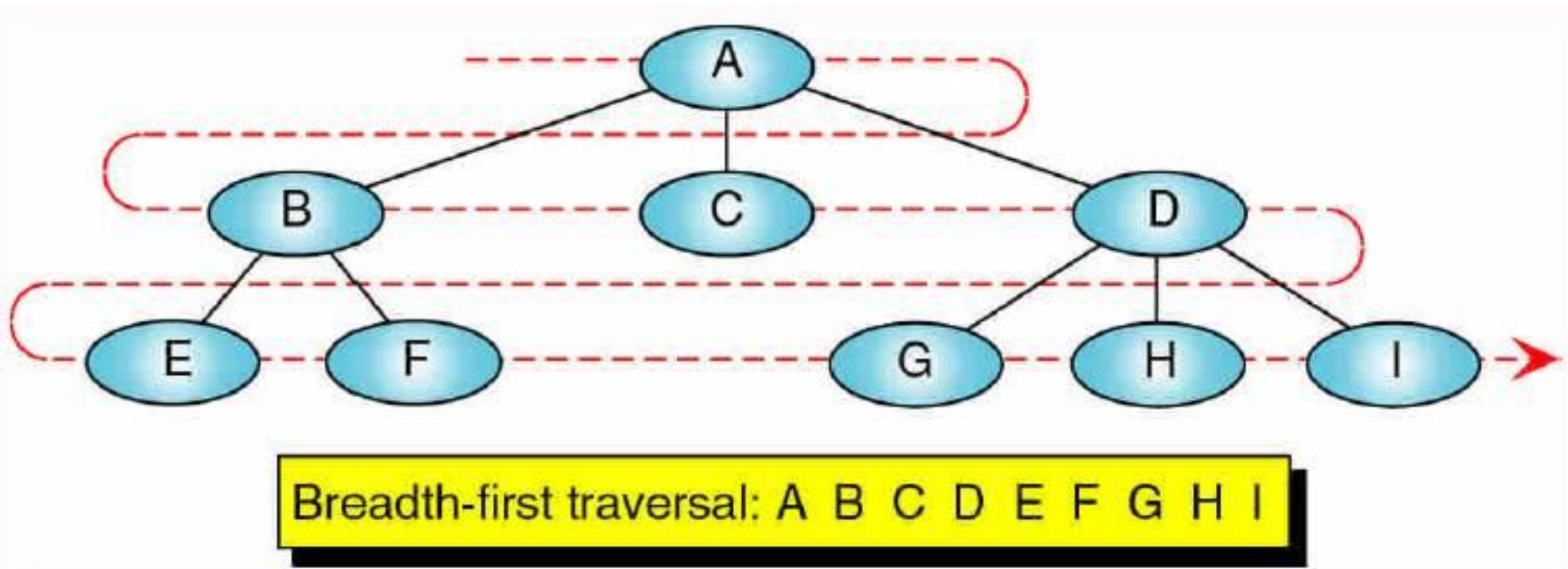
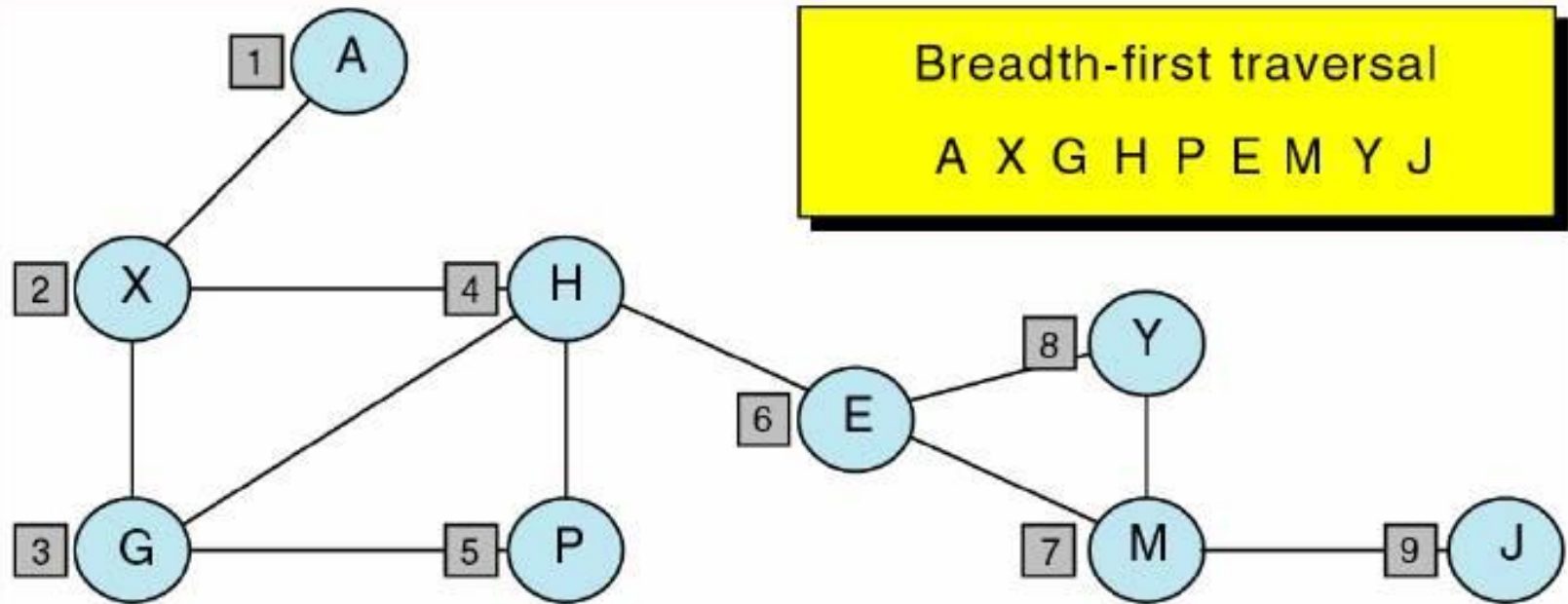


Fig.11-12 BFT with a Queue



(a) The graph

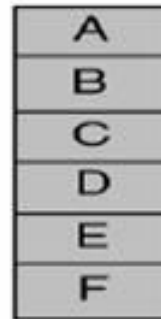
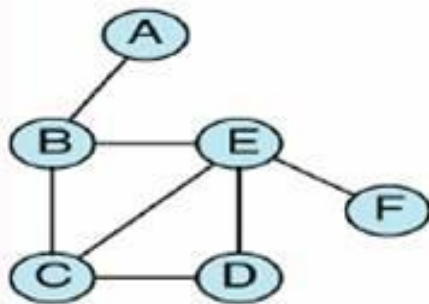


(b) Queue contents

Graph Storage Structures

- Need to store two sets
 - Vertices
 - Edges
- Data structures
 - Adjacent matrix
 - Adjacent list

Fig.11-13 Adjacent Matrix

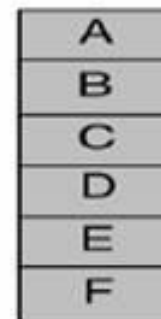
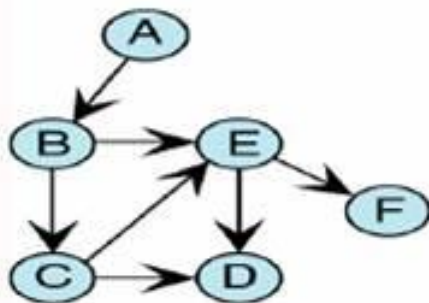


Vertex vector

	A	B	C	D	E	F
A	0	1	0	0	0	0
B	1	0	1	0	1	0
C	0	1	0	1	1	0
D	0	0	1	0	1	0
E	0	1	1	1	0	1
F	0	0	0	0	1	0

Adjacency matrix

(a) Adjacency matrix for non-directed graph



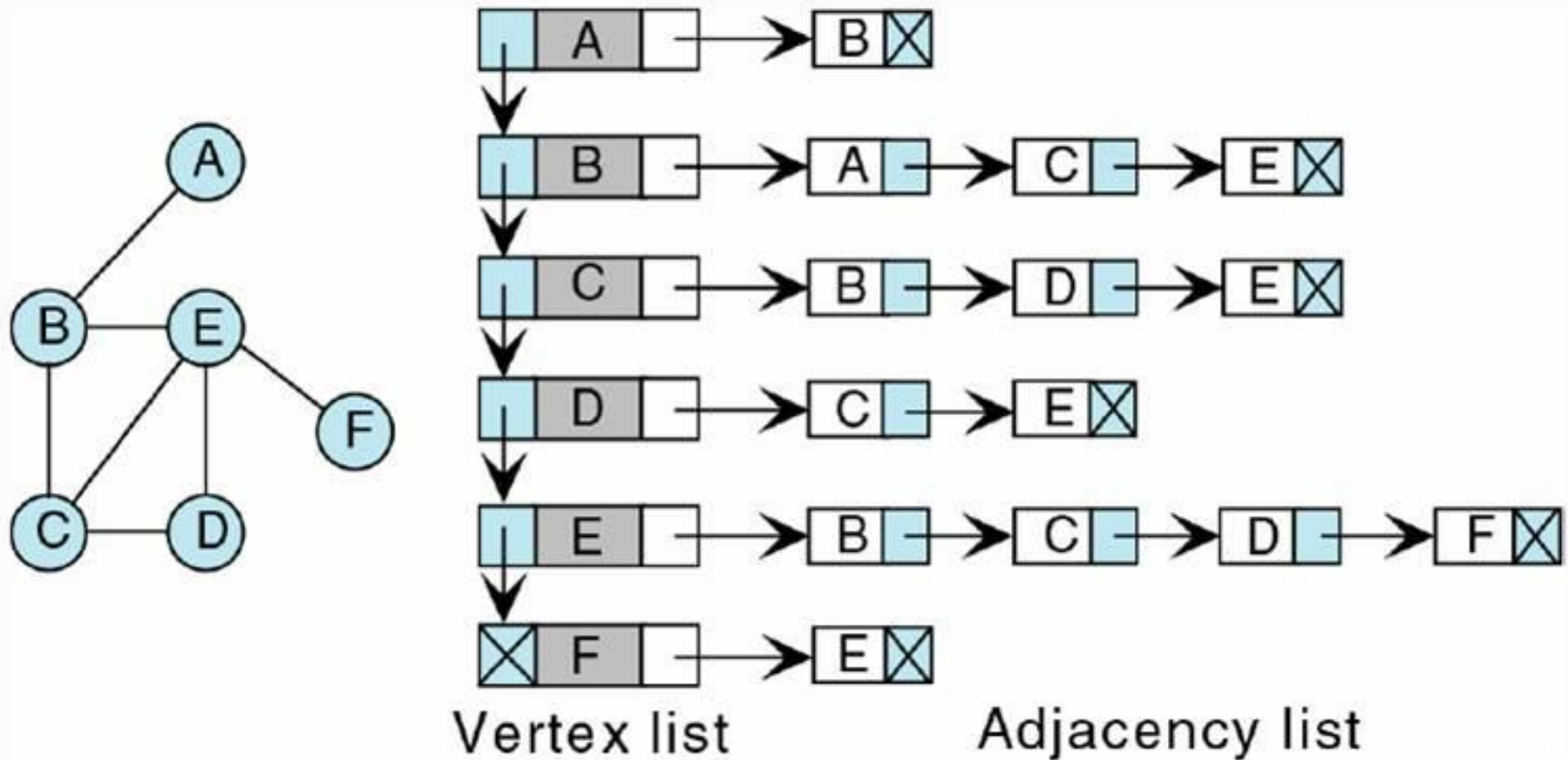
Vertex vector

	A	B	C	D	E	F
A	0	1	0	0	0	0
B	0	0	1	0	1	0
C	0	0	0	1	1	0
D	0	0	0	0	0	0
E	0	0	0	1	0	1
F	0	0	0	0	0	0

Adjacency matrix

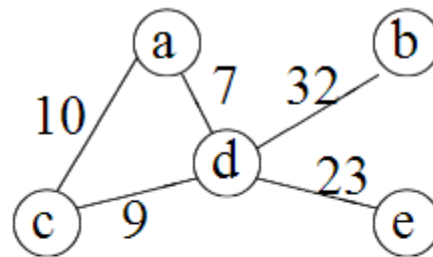
(a) Adjacency matrix for directed graph

Fig.11-14 Adjacent List

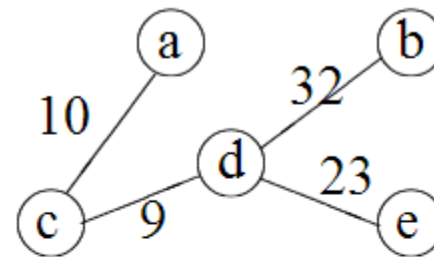


Weighted Graphs

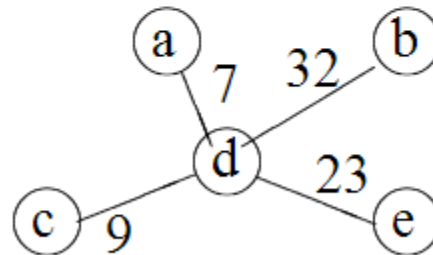
- A weighted graph is a graph, in which each edge has a weight (some real number).
- **Weight of a Graph:** The sum of the weights of all edges



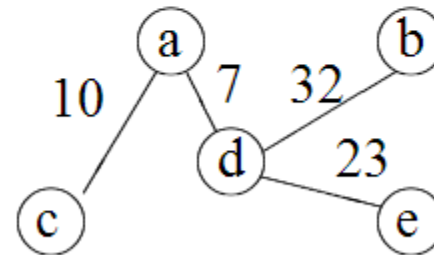
weighted graph



Tree 1. $w=74$



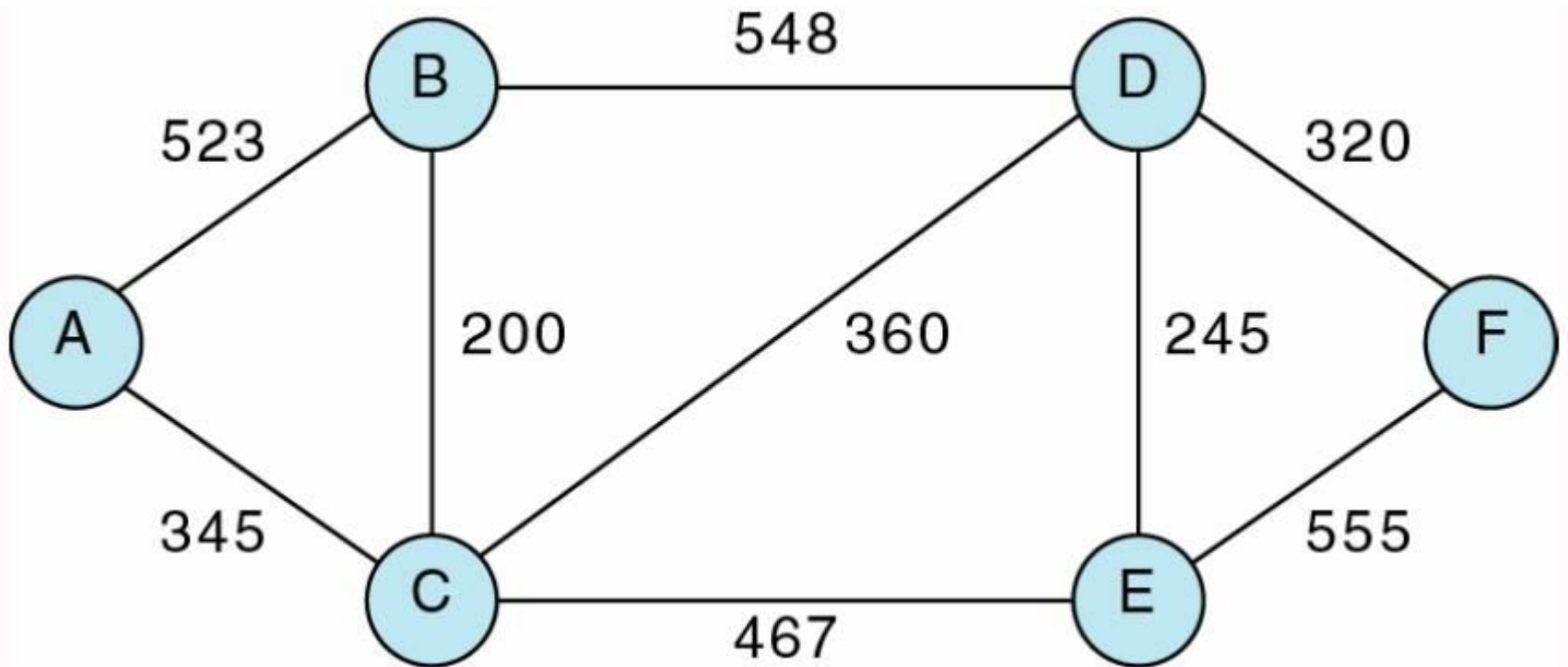
Tree 2, $w=71$



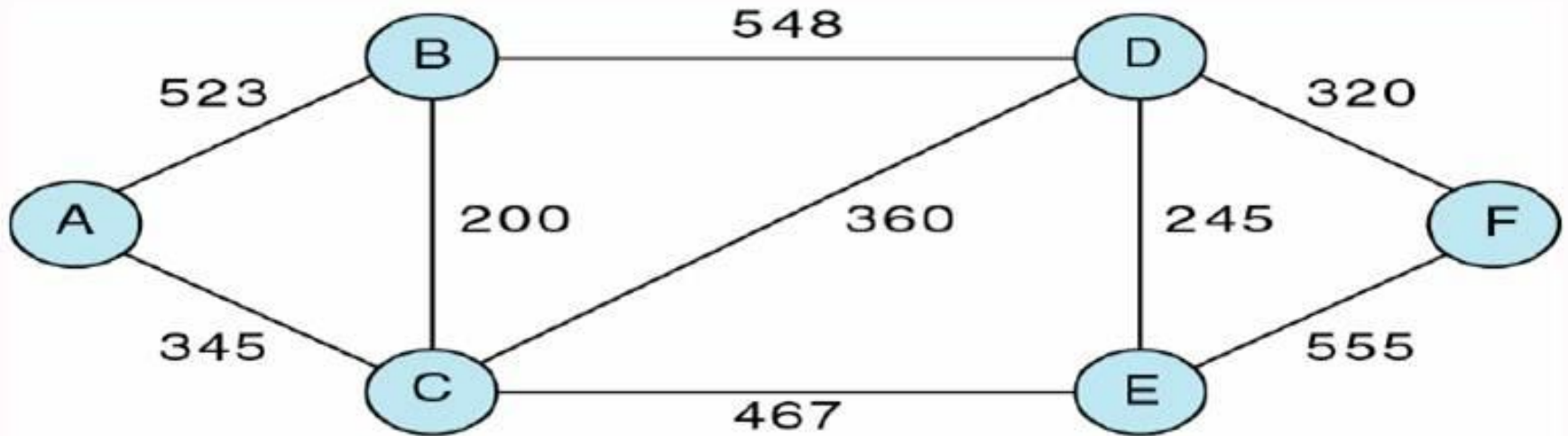
Tree 3, $w=72$

Networks

- Graph with weighted arcs –Weighted Graph
- Fig.11-18



Adjacency Matrix for Weighted Graph



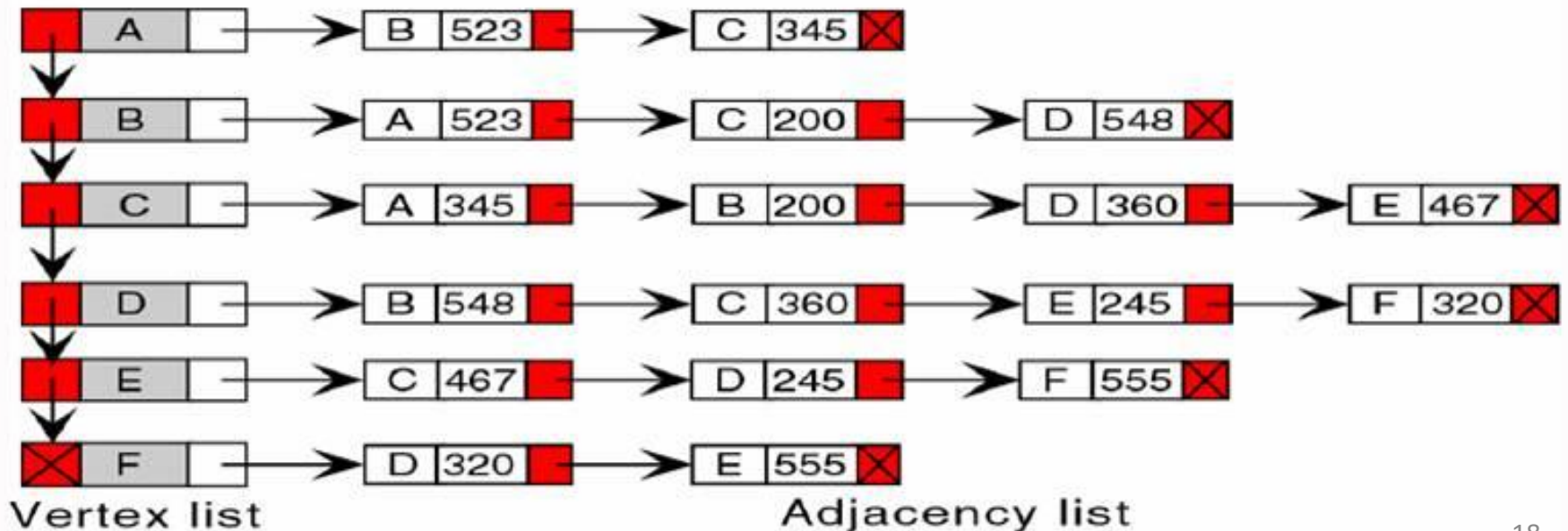
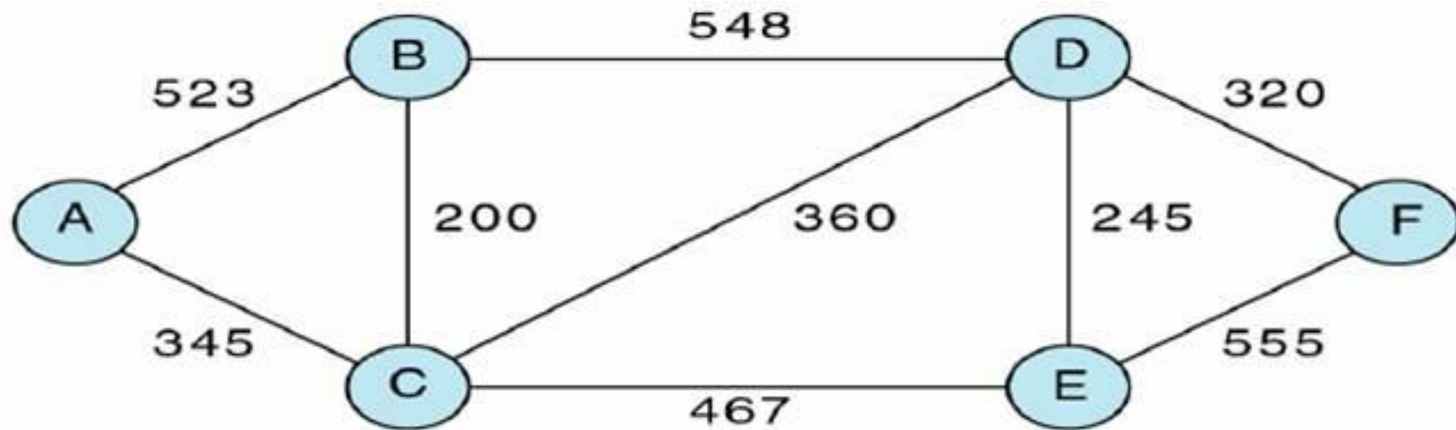
A
B
C
D
E
F

Vertex vector

	A	B	C	D	E	F
A	0	523	345	0	0	0
B	523	0	200	548	0	0
C	345	200	0	360	467	0
D	0	548	360	0	245	320
E	0	0	467	245	0	555
F	0	0	0	320	555	0

Adjacency matrix

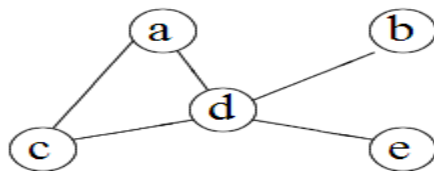
Adjacency List Representation of Weighted Graph



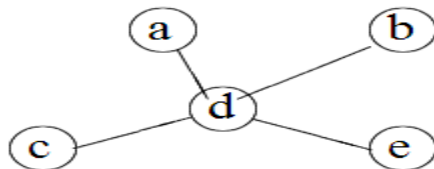
Minimum Spanning Tree

- Spanning Tree

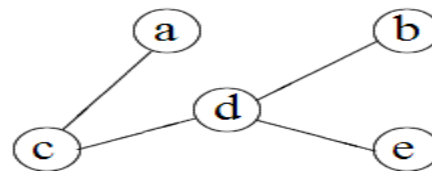
- A spanning tree of a simple graph G is a subgraph of G that is a tree containing every vertex of G .
- We can construct a spanning tree using *breadth-first search* or *depth-first search*



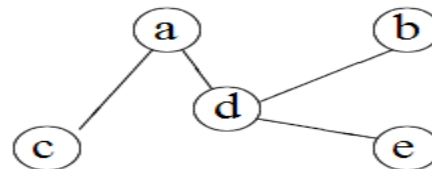
Graph



spanning tree 2



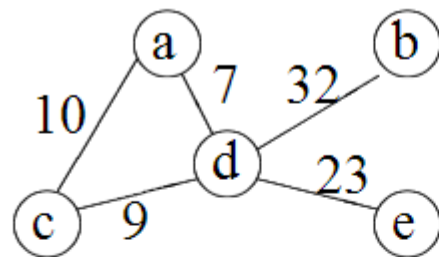
spanning tree 1



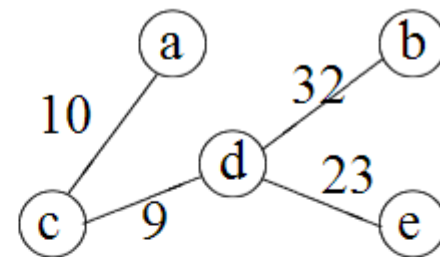
spanning tree 3

Minimum Spanning Tree

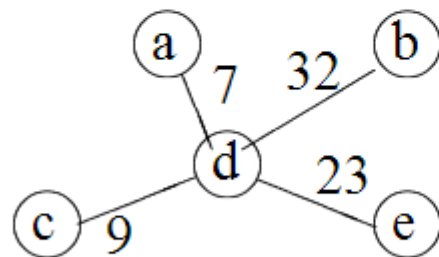
- A Minimum Spanning Tree of an undirected connected weighted graph is a spanning tree of minimum weight (among all spanning trees).



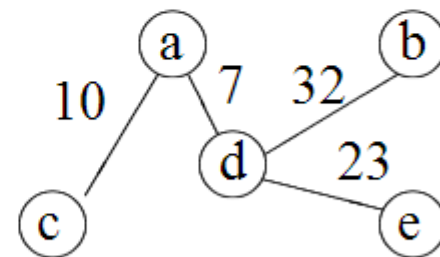
weighted graph



Tree 1. $w=74$



Tree 2, $w=71$

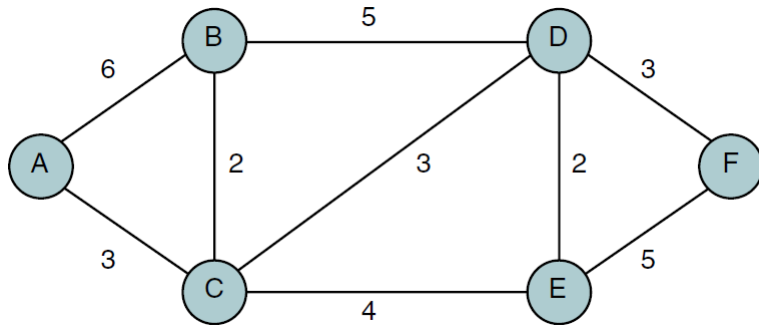


Tree 3, $w=72$

Minimum spanning tree

Minimum Spanning Tree (Prim's Algorithm)

- **Process:** Select edges with the minimal value to a vertex not currently in the tree repeatedly and insert them into the tree.

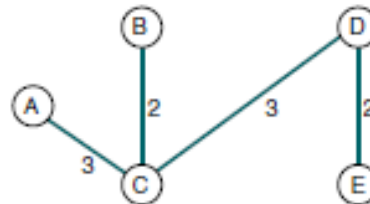


(a)

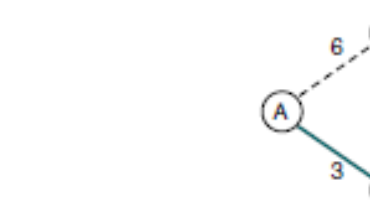
Insert first vertex



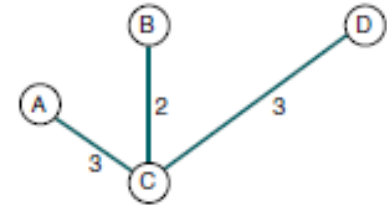
(c) Insert edge BC



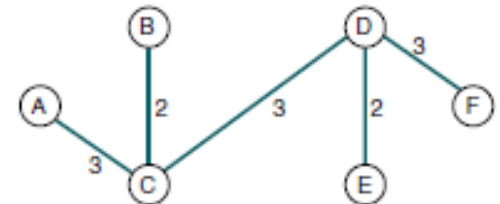
(e) Insert edge DE



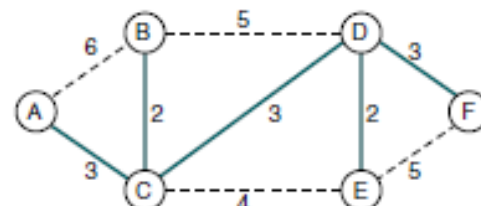
(b) Insert edge AC



(d) Insert edge CD



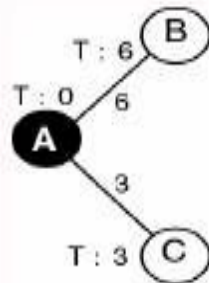
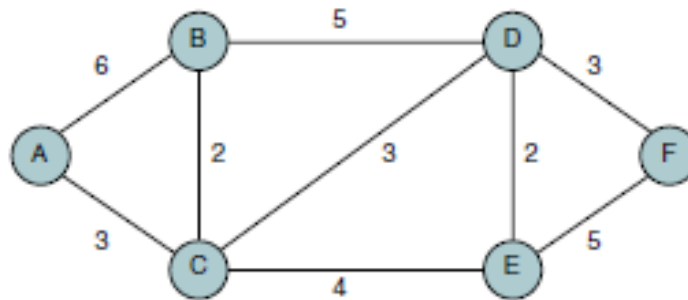
(f) Insert edge DF



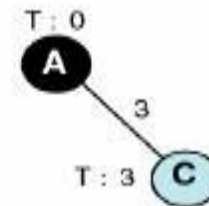
(g) Final tree in the graph

Shortest Path Algorithm: Dijkstra algorithm

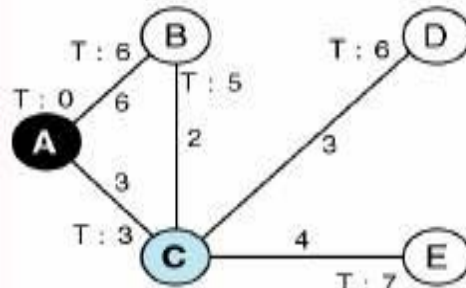
- Finding the shortest path between two vertices in a network
- Application: to find the least expensive route between home and our destination
- The Dijkstra algorithm is used to find the shortest path between any two nodes in a graph.
- Steps
 1. Insert the first vertex into the tree
 2. From every vertex already in the tree
 - i. Examine the total path length to all adjacent vertices not in the tree
 - ii. Select the edge with the minimum total path weight and insert it into the tree
 3. Repeat Step 2 until all vertices are in the tree



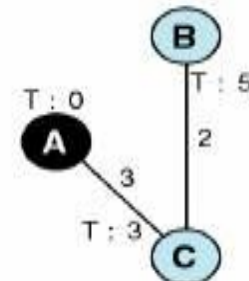
(a1) Possible paths from A1



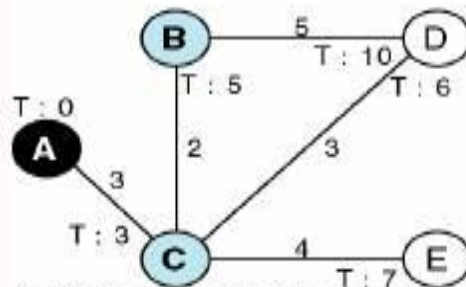
(a2) Tree after insertion of C



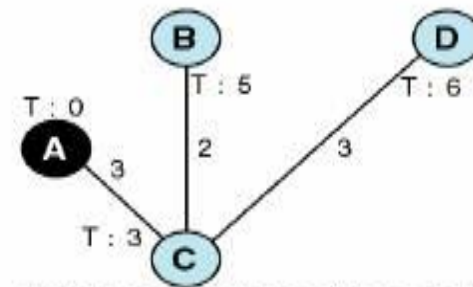
(b1) Possible paths from A and C



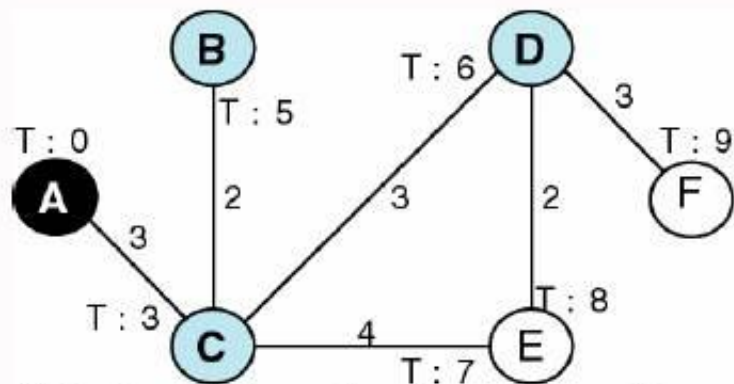
(b2) Tree after insertion of B



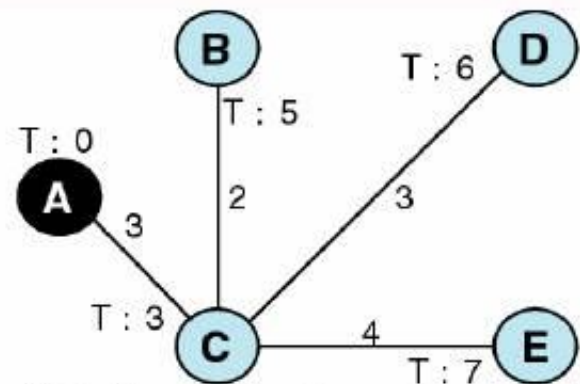
(c1) Possible paths from B and C



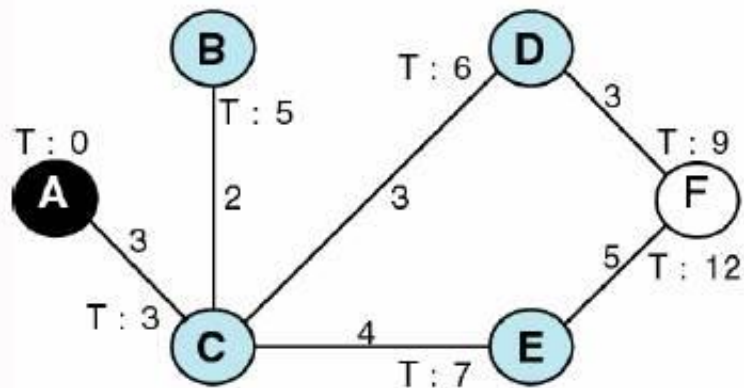
(c2) Tree after insertion of D



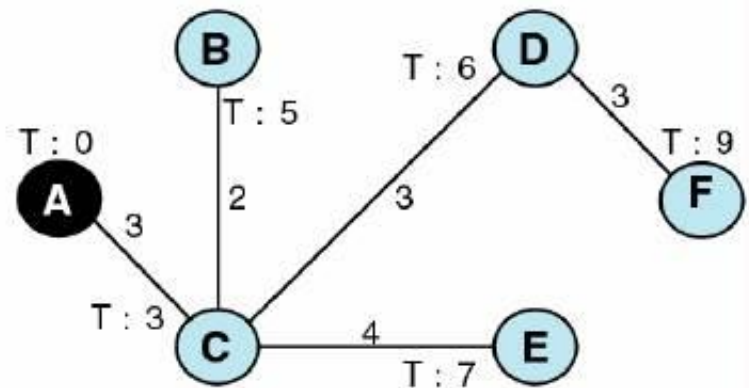
(d1) Possible paths from C and D



(d2) Tree after insertion of E



(e1) Possible paths from D and E



(e2) Tree after insertion of F

T : n Total path length from A to node

Exercises

Chapter 11 (Graphs)

Exercises: : 4,5, 9,10,11,12, 13,

Source: Data Structures: A Pseudocode Approach Using C, 2nd ed